

# Polygonal Rotor Networks and Higher-Dimensional Orbit Representations

Notes Toward a Non-Gradient Phase-Based Computing Architecture

|        |                                    |
|--------|------------------------------------|
| Status | Exploratory research notes         |
| Date   | September 2026                     |
| Author | Adam Sohnen et al. (working draft) |

## Abstract

This work began with a simple question:

*Can cyclic polygonal structures serve as computational units whose internal phase states solve problems through geometric search rather than gradient descent?*

Several architectures were explored: a continuous harmonic Polygonal Rotor Network (PRN), a discrete Hidden-37-derived admissibility rotor, hybrid arithmetic-phase systems, and high-dimensional orbit-preserving rotor controllers.

Several negative results were obtained: certain proposed slack functions were phase invariant; some rotor search strategies explored only tiny subsets of phase space; and local phase search often failed to outperform random search.

*The central positive observation is that performance improved markedly when the full polygonal orbit representation was preserved instead of collapsing each node to one or two scalar dimensions.*

The strongest evidence for this effect appeared not in XOR, but in CartPole control experiments.

## 1. Motivation

Most neural systems reduce node state immediately to a scalar,  $y = f(x)$ , or at most a low-dimensional activation vector. The original Polygonal Rotor concept proposed that nodes possess an internal cyclic geometry,  $\rho$  in  $\mathbb{Z}_k$ , where computation occurs through phase evolution.

*Useful computational structure may exist in the entire orbit of cyclic states, not merely in a scalar projection of that orbit.*

## 2. Early Architectures

### 2.1 Continuous Harmonic PRN

Each polygon carried phase  $\theta$ , velocity  $\omega$ , direction  $\sigma$ , and vertex values  $v_i$ . Output was generated through a harmonic projection:

$$f(\theta) = \sum_m v_m \cos(2\pi m/k - \theta)$$

Subsequent analysis showed that this collapses to  $C \cos(\theta) + S \sin(\theta)$ , where  $C$  and  $S$  are the first harmonic coefficients. Thus all polygon orders become equivalent two-dimensional oscillators. This was an important negative result.

## 2.2 Hidden-37 Rotor

The discrete architecture was inspired by admissible cyclic polygons, with activation derived from:

$$2 \max_r \text{rot}(B, r) < R_k s$$

- Advantages: exact arithmetic meaning, no gradient descent, finite state space, and exhaustive verification.
- Disadvantage: the earliest version discarded metric information.

## 3. Phase-Invariance Discovery

A critical failure mode was identified. Consider the normalized full-orbit slack:

$$S(B, \rho) = [\sum_r \text{rot}(B, r) - 2 \max_r \text{rot}(B, r)] / \sum_r \text{rot}(B, r)$$

Because every phase visits all rotations exactly once,  $S(B, \rho) = S(B, 0)$  for every  $\rho$ . The rotor could move while the computation remained unchanged. The system was therefore mathematically incapable of phase learning, eliminating several early variants.

## 4. Phase-Sensitive Repair

A repaired slack metric was introduced. Rather than maximize over all rotations, phase selects a distinguished side:

$$S_{\rho} = [\sum_j R_j - 2 R_{\rho}] / \sum_j R_j$$

Phase now determines which side is selected, making rotor state observable. This repair produced XOR-capable architectures and allowed meaningful phase-search experiments.

## 5. XOR Experiments

For rotor orders (3,4,5,6,3), the joint phase space contains  $3 \times 4 \times 5 \times 6 \times 3 = 1080$  configurations.

Exhaustive enumeration showed that many architectures contain exact XOR states, but random restart was difficult to outperform and local phase search frequently became trapped. Tested methods included lockstep rotation, coprime stride schedules, coordinate descent, tabu search, and margin search. None consistently exceeded random search.

This suggested that XOR may be too small and too discrete to expose an advantage of rotor geometry.

## 6. Dimensionality Hypothesis

A pattern emerged: every unsuccessful architecture compressed node state to very low dimension. Hidden-37 and continuous slack reduced the node to one scalar; the harmonic PRN reduced it to two coefficients.

*Rotor computation may fail when phase structure is projected into too few dimensions.*

## 7. Full Orbit Representation

Instead of retaining only one scalar, each node preserves the entire cyclic slack orbit:

$$\Phi_{\rho} = [s_{\rho}, s_{(\rho+1)}, \dots, s_{(\rho+k-1)}]$$

A triangle is represented in  $R^3$ , a square in  $R^4$ , a pentagon in  $R^5$ , and so on. Rotors interact through orbit alignment rather than scalar activation.

## 8. Common-Dimensional Lift

To combine heterogeneous polygons, each orbit was periodically lifted into  $\text{LCM}(3,4,5,6) = 60$  dimensions. No coordinates were learned; only phase changed.

## 9. High-Dimensional XOR Results

| Representation  | XOR-capable architectures | Median solution states |
|-----------------|---------------------------|------------------------|
| Scalar variants | Approximately 44%-48%     | Approximately 20-50    |
| Full orbit      | Approximately 64%         | Approximately 120      |

Preserving orbit structure dramatically enlarged observed solution basins in the exploratory XOR experiments.

## 10. CartPole Experiment

A more meaningful dynamical benchmark was then constructed using fixed architecture, fixed projections, no learned weights, and phase-only policy variation. Each architecture contained 1080 possible phase configurations and was evaluated from multiple initial conditions.

### Scalar Controller

Each rotor contributes one scalar slack coordinate,  $s_{\rho}$ .

### Full Orbit Controller

Each rotor contributes a lifted 60-dimensional orbit  $\Phi_{\rho}$ . The hidden state is  $H = \sum_i w_i \Phi_i$ , and the output action is determined by orbit alignment.

## 11. Exploratory CartPole Findings

| Controller     | Median best survival    | Perfect episodes              |
|----------------|-------------------------|-------------------------------|
| Scalar slack   | Approximately 200 steps | Observed less often           |
| Full orbit 60D | Approximately 263 steps | Observed about twice as often |

Head-to-head exploratory comparisons favored the full-orbit representation more frequently than the scalar representation. Unlike XOR, CartPole has continuous dynamics, long-horizon effects, multiple attractors, and meaningful policy neighborhoods. The full-orbit representation appears to exploit this structure.

## 12. Tentative Conclusion

*Rotor phase alone does not automatically produce useful computation. However, preserving the full cyclic orbit appears substantially more effective than collapsing that orbit into one or two dimensions.*

The observed benefits included increased numbers of viable XOR solutions, larger solution basins, improved exploratory CartPole policy quality, and more informative medium-budget phase search. These results do not establish superiority over modern reinforcement learning or a new learning paradigm. They suggest that cyclic arithmetic orbits may carry useful computational information that disappears under low-dimensional projection.

## Future Work

- Larger, fully logged CartPole study with paired seeds and equal rollout budgets.
- Acrobot, double pendulum, inverted double cart, and inverted quadruple pendulum.
- Explicit comparison against random search, evolutionary search, CMA-ES, PPO, and SAC.
- Higher-harmonic orbit embeddings and orbit-alignment kernels.
- Continuous phase interpolation.
- Rotor memory and adaptive phase dynamics.

## Appendix A. Core Full-Orbit Lift

```
def lift(v, phase):
    return np.tile(
        np.roll(v, int(phase) % len(v)),
        60 // len(v)
    )
```

## Appendix B. Orbit Construction

```
vals = np.array(
    [rot_val(digits, r) for r in range(k)],
    dtype=float
)

perimeter = vals.sum()
slack = (perimeter - 2 * vals) / perimeter
slack -= slack.mean()
orbit = slack / np.linalg.norm(slack)
```

## Appendix C. CartPole Dynamics

```
def cp_step(state, action):
    x, x_dot, theta, theta_dot = state
    force = 10.0 if action else -10.0
    costheta = np.cos(theta)
    sintheta = np.sin(theta)

    temp = (
        force + 0.05 * theta_dot**2 * sintheta
    ) / 1.1

    thetaacc = (
        9.8 * sintheta - costheta * temp
    ) / (
        0.5 * (4 / 3 - 0.1 * costheta**2 / 1.1)
    )

    xacc = temp - 0.05 * thetaacc * costheta / 1.1

    return np.array([
        x + 0.02 * x_dot,
        x_dot + 0.02 * xacc,
        theta + 0.02 * theta_dot,
        theta_dot + 0.02 * thetaacc
    ])
```

## Appendix D. Full-Orbit Controller

```
H = np.zeros(60)

for i, k in enumerate(rotor_orders):
    offset = phase_offset(observation, i, k)
    H += (i + 1) * lift(
        orbit_vectors[i],
        rho[i] + offset
    )

score = (
    H * lift(output_orbit, rho_out)
) @ readout

action = score > 0
```

## Appendix E. Scalar Controller

```

score = 0.0

for i, k in enumerate(rotor_orders):
    offset = phase_offset(observation, i, k)
    score += (i + 1) * orbit_vectors[i][
        (rho[i] + offset) % k
    ]

score *= 1 + 0.25 * output_orbit[rho_out]
action = score > 0

```

## Appendix F. Complete Reproducible Benchmark Script

```

import itertools
import numpy as np

ROTOR_ORDERS = (3, 4, 5, 6, 3)
LIFT_DIM = 60
PHASES = np.array(list(itertools.product(
    *[range(k) for k in ROTOR_ORDERS]
)))

def rot_val(digits, r):
    r %= len(digits)
    shifted = digits[r:] + digits[:r]
    value = 0
    for digit in shifted:
        value = 10 * value + digit
    return value

def make_architecture(seed):
    rng = np.random.RandomState(seed)
    orbit_vectors = []

    for k in ROTOR_ORDERS:
        digits = rng.randint(0, 10, size=k).tolist()
        rotations = np.array([
            rot_val(digits, r) for r in range(k)
        ], dtype=float)
        perimeter = rotations.sum()
        slack = (
            (perimeter - 2 * rotations) / perimeter
            if perimeter else np.zeros(k)
        )
        slack -= slack.mean()
        norm = np.linalg.norm(slack)
        orbit_vectors.append(slack / norm if norm else slack)

    input_projection = rng.normal(size=(4, 4))
    readout = rng.choice([-1.0, 1.0], LIFT_DIM)
    return orbit_vectors, input_projection, readout

def lift(vector, phase):
    rotated = np.roll(vector, int(phase) % len(vector))
    return np.tile(rotated, LIFT_DIM // len(vector))

def cartpole_step(state, action):
    x, x_dot, theta, theta_dot = state
    force = 10.0 if action else -10.0
    costheta = np.cos(theta)
    sintheta = np.sin(theta)

    temp = (force + 0.05 * theta_dot**2 * sintheta) / 1.1
    theta_acc = (
        9.8 * sintheta - costheta * temp
    ) / (
        0.5 * (4.0 / 3.0 - 0.1 * costheta**2 / 1.1)
    )

```

```

    )
    x_acc = temp - 0.05 * theta_acc * cosheta / 1.1

    return np.array([
        x + 0.02 * x_dot,
        x_dot + 0.02 * x_acc,
        theta + 0.02 * theta_dot,
        theta_dot + 0.02 * theta_acc,
    ])

def normalized_observation(state):
    return np.array([
        state[0] / 2.4,
        state[1] / 3.0,
        state[2] / 0.20944,
        state[3] / 3.0,
    ])

def phase_offset(observation, row, k, projection):
    drive = np.tanh(projection[row] @ observation)
    return int(np.floor((drive + 1.0) * k / 2.0)) % k

def controller_action(state, phases, architecture, full_orbit=True):
    vectors, projection, readout = architecture
    obs = normalized_observation(state)

    if full_orbit:
        hidden = np.zeros(LIFT_DIM)
        for i, k in enumerate(ROTOR_ORDERS[:4]):
            offset = phase_offset(obs, i, k, projection)
            hidden += (i + 1) * lift(
                vectors[i], phases[i] + offset
            )
        score = (
            hidden * lift(vectors[4], phases[4])
        ) @ readout
    else:
        score = 0.0
        for i, k in enumerate(ROTOR_ORDERS[:4]):
            offset = phase_offset(obs, i, k, projection)
            score += (i + 1) * vectors[i][
                (phases[i] + offset) % k
            ]
        score *= 1.0 + 0.25 * vectors[4][phases[4]]

    return int(score > 0.0)

def run_episode(phases, architecture, full_orbit, initial_state,
                max_steps=500):
    state = initial_state.copy()
    for step in range(1, max_steps + 1):
        action = controller_action(
            state, phases, architecture, full_orbit
        )
        state = cartpole_step(state, action)
        if abs(state[0]) > 2.4 or abs(state[2]) > 0.20944:
            return step
    return max_steps

def evaluate_policy(phases, architecture, full_orbit, starts):
    return np.mean([
        run_episode(phases, architecture, full_orbit, start)
        for start in starts
    ])

def main():

```

```

search_rng = np.random.RandomState(20260926)
architectures = 30
policies_per_architecture = 100
starts_per_policy = 10

results = []
for seed in range(architectures):
    architecture = make_architecture(seed)
    start_rng = np.random.RandomState(1000 + seed)
    starts = [
        start_rng.uniform(-0.05, 0.05, 4)
        for _ in range(starts_per_policy)
    ]

    best_full = 0.0
    best_scalar = 0.0

    for _ in range(policies_per_architecture):
        phases = tuple(
            search_rng.randint(k) for k in ROTOR_ORDERS
        )
        best_full = max(
            best_full,
            evaluate_policy(phases, architecture, True, starts)
        )
        best_scalar = max(
            best_scalar,
            evaluate_policy(phases, architecture, False, starts)
        )

    results.append((best_full, best_scalar))

full = np.array([x[0] for x in results])
scalar = np.array([x[1] for x in results])

print('Full-orbit median:', np.median(full))
print('Scalar median:', np.median(scalar))
print('Full better:', np.sum(full > scalar))
print('Ties:', np.sum(full == scalar))
print('Scalar better:', np.sum(full < scalar))

if __name__ == '__main__':
    main()

```

**Research note:** These results are exploratory and should be treated as a research diary rather than a finished paper. The most important finding is not that rotor networks are established, but that orbit preservation appears more consequential than any particular rotor learning rule examined so far.